

Практика 6. Классы типов

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Полиморфные функции в Haskell
- ▶ Структуры данных в Haskell, полиморфные структуры данных
- NB Использование структур данных и `newtype` для контроля за выполнением инвариантов предметной области¹

$$\boxed{\sigma_1} \rightarrow \sigma_2 \rightarrow \cdots \rightarrow \sigma_{k-1} \rightarrow \boxed{\sigma_k}$$

- + Классы типов
- + Некоторые стандартные классы типов языка Haskell

¹<https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/>

todo Как определён **Either**?

todo Какой будет результат выполнения следующего выражения?

```
case B 1 2 of
  A x -> x + 3
  B x y -> x + y
  C -> 42
```

todo Приведите пример терма с типом **Maybe**

todo Какой тип у **Left** 42?

1. Простые классы типов
2. Классы типов высших кайндов
3. Реализация классов типов в GHC (⊕)
4. Материалы (☀)

Содержание

1. Простые классы типов
2. Классы типов высших кайндов
3. Реализация классов типов в GHC (⊕)
4. Материалы (☀)

Классы типов как механизм перегрузки

- **Перегрузка (overloading)** — из функций с одинаковыми именами и разными типами выбирается подходящая

```
string show(int x) { ... }
string show(data d) { ... }
```

- Классы типов требуют задекларировать перегружаемую функцию и варьируемый типовой параметр явно

```
class Show a where
  show :: a -> String
```

- Для каждого типа пишется по инстансу

```
instance Show Int where
  show :: Int -> String
  show x = ...
```

```
instance Show Data where
  show :: Data -> String
  show d = ...
```

- В отличие от классической перегрузки, классы типов дружат с параметрическим полиморфизмом

```
showWithPrefix :: Show a => a -> String -> String
```

Задачи

todo Реализуйте `Show` для следующего типа

```
data LogLevel = Error | Warning | Debug | Info
```

todo Реализуйте `Show` для пары

todo Придумайте класс для парсинга данных `MyRead`

```
⚡ instance (Show a, Show b) => Show (a, b) where  
    show (x, y) = "(" ++ show x ++ "," ++ show y ++ ")"
```

```
⚡ class MyRead a where  
    readMaybe :: String -> Maybe a  
    -- варьируемый типовой параметр может использоваться где угодно
```

Ещё задачи

todo Реализуйте функцию `allValues`, перебирающую все значения типа

todo Реализуйте проверку равенства функций

```
⚡ allValues :: (Enum a, Bound a) => [a]  
allValues = [minBound .. maxBound]
```

```
⚡ instance (Enum a, Bound a, Eq b) => Eq (a -> b) where  
  (==) :: (a -> b) -> (a -> b) -> Bool  
  f == g = map f allValues == map g allValues
```

Содержание

1. Простые классы типов
2. Классы типов высших кайндов
3. Реализация классов типов в GHC (⊕)
4. Материалы (☀)

Классы высших кайндов и функторы

- ▶ Рассмотренные ранее классы типов ожидают типы кайнда $*$ ²

```
class Ord (a :: *) where
  (<) :: a -> a -> Bool
```

- ▶ Для абстрагирования по коллекциям нужны типы высших кайндов

```
class Functor (f :: * -> *) where
  fmap :: (a -> b) -> f a -> f b
```

todo Реализуйте `Functor` для `Maybe` и списков

```
instance Functor Maybe where
  fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap f mb = case mb of Nothing -> Nothing; Just x -> Just (f x)
instance Functor [] where
  fmap :: (a -> b) -> [a] -> [b] -- или [a], [b]
  fmap = map
```

²В современном Haskell вместо звёздочек принято использовать `Data.Kind.Type`.

Частичное применение типовых конструкторов

```
todo ghci> :k (,)
```

```
todo ghci> :k (->)
```

```
todo ghci> :k (,) Int
```

todo Реализуйте функтор для пары

```
⚡ (,) :: * -> * -> *
```

```
⚡ (->) :: * -> * -> *
```

```
⚡ (,) Int :: * -> *
```

```
⚡ instance Functor ((,) e) where -- (,) :: * -> * -> *
    fmap :: (a -> b) -> ((,) e) a -> ((,) e) b
    --   :: (a -> b) -> (e, a)   -> (e, b)
    fmap f (w, x) = (w, f x)
```

Содержание

1. Простые классы типов
2. Классы типов высших кайндов
3. Реализация классов типов в GHC (☼)
4. Материалы (☼)

Полиморфная сортировка: базовые классы типов

- ▶ Хотим одной функцией сортировать списки элементов различных типов

```
sort :: [a] -> [a]
```

- ▶ Нужно сравнивать элементы, по-разному для каждого типа

⇒ Будем принимать функцию “меньше” для нужного типа

```
sort :: (a -> a -> Bool) -> [a] -> [a]
```

- ▶ Потенциально функций может быть много, заведём рекорд

```
data MyOrd a = MyOrd { less :: a -> a -> Bool }
```

```
sort :: MyOrd a -> [a] -> [a]
```

- ▶ Заводим экземпляр для конкретного типа

```
intMyOrd :: MyOrd Int
```

```
intMyOrd = MyOrd { less = (<) }
```

- ▶ Вызов теперь выглядит так

```
GHCI> sort intMyOrd [3, 2, 1]
```

```
[1, 2, 3]
```

Свойство упорядоченности

Через `data`

1. Определение словаря функций

```
data MyOrd a = MyOrd
    { less :: a -> a -> Bool }
```

2. Экземпляр словаря для конкретного типа

► Именованное значение

```
intMyOrd :: MyOrd Int
intMyOrd = MyOrd { less = (<) }
```

3. Явный параметр функции

```
sort :: MyOrd a -> [a] -> [a]
```

4. Передаётся пользователем

```
test = sort intMyOrd [3, 2, 1]
```

Через `class`

1. Определение класса типов

```
class MyOrd a where
    less :: a -> a -> Bool
```

2. Объявление типа представителем класса типов

► Не имеет имени

```
instance MyOrd Int where
    less = (<)
```

3. Неявный параметр функции

```
sort :: MyOrd a => [a] -> [a]
```

4. Передаётся компилятором¹

```
test = sort [3, 2, 1]
```

¹Нужен импорт модуля, даже пустой, с определением нужного инстанса: `import Module ()`.

Сортировка полиморфных списков: расширение инстансов

- ▶ Хотим объявить список упорядоченным, если упорядочены его элементы
- ▶ Должный завести функцию, которая даст реализацию для списка по реализации для его элементов:

```
listMyOrd :: MyOrd a -> MyOrd [a]
listMyOrd aOrd = MyOrd
  { less = fix $ \rec xs ys -> case (xs, ys) of
    -- ...
    (x:xs, y:ys) | aOrd x y = True
    -- ...
  }
```

- ▶ Теперь можем сортировать списки списков!

```
test = sort (listMyOrd intMyOrd) [[3, 2], [], [1]]
```

```
listMyOrd :: MyOrd a -> MyOrd [a]           instance MyOrd a => MyOrd [a] where
listMyOrd aOrd = MyOrd { less = -- ...      less = -- ...
```

- ▶ Всего один неявный параметр и механизм его вывода даёт столько могущества
- ▶ Однако вывод инстансов — очень не простое дело на стороне компилятора

Расширение классов типов

- ▶ Иногда одно свойство влечёт другое как более слабое
- ▶ В Haskell инстанс расширенного класса типов просто требует наличия инстанса базового
- ▶ Пример: частичный порядок (работает не для каждой пары элементов)

```
class MyPartialOrd a where
  mbLess :: a -> a -> Maybe Bool
```

- ▶ Частичный порядок обязателен для полного^{3,4}

```
class MyPartialOrd a => MyOrd a where
  less :: a -> a -> Bool
```

- ▶ При объявлении типа представителем базового класса типов пользоваться объявлением для расширенного

```
instance MyPartialOrd Int where
  mbLess = Just . less
```

³Например, числа с плавающей точкой по стандарту IEEE 754 упорядочены только частично (NaN).

⁴В Haskell так не сделано, но сделано в Rust. Менее удобно, но более безопасно.

Содержание

1. Простые классы типов
2. Классы типов высших кайндов
3. Реализация классов типов в GHC (⊕)
4. Материалы (☀)

Похожие концепции в других языках (☁️)

- ▶ А ля классы типов, диспатч по нулевому аргументу функций
 - e.g. Rust traits
 - e.g. Swift protocols
 - e.g. Scala traits + implicits
- ▶ Контроль за инстанциацией шаблонов
 - e.g. C++ 20 концепты
- ▶ Неявные ресиверы
 - e.g. Kotlin context receivers
- ▶ Шаблон visitor для динамического диспатча по нескольким аргументам одновременно
- ▶ Мультиметоды
 - e.g. Julia

- ▶ [Typeclassopedia](#): шпаргалка по зоопарку классов типов

- ▶ Tagless-Final Style
- ▶ Олег Нижников — Современное ФП с Tagless Final
- ▶ The Death of Tagless Final by John A. De Goes
- ▶ Review: Codata in Action
- ▶ Tagless Final style by Oleg
- ▶ Lecture notes on Tagless Final
- ▶ Wadler, Philip, and Stephen Blott. "How to make ad-hoc polymorphism less ad hoc." *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 1989.